

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: CSA14

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO1: Apply lexical analysis for token specification and recognition using LEX tool (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 3

Total Marks:50

Annexure A

ERROR ANALYSIS TASK

Code of Conduct:

I ANIKSHA EVANJALIN M (192521308) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ANIKSHA

Annexure A

ERROR ANALYSIS TASK

Q1. The following program is intended to perform simple arithmetic operations, function calls, array access, and looping constructs. However, the code contains **multiple errors introduced at different phases of compilation**, including **lexical, syntax, semantic, logical, and runtime errors**.

Given Program

```
#include <stdio.h>
```

```
int sumArray(int arr[], int n) {  
    int i, sum = 0;  
    for(i = 0; i <= n; i++) {  
        sum = sum + arr[i];  
    }  
    return sum;  
}
```

```
int main() {  
    int arr[5] = {1,2,3,4,5}  
    int total;
```

```
    total = sumArray(arr);
```

```
    if(total == 15) {  
        printf("Correct Sum\n")  
    }
```

```
    int a = 10, b = 0;  
    int c = a / b;
```

```
    int *p;  
    *p = 20;
```

```
    return 0;  
}
```

Tasks:

(a) Error Identification (Analysis Level)

1. Identify all the errors present in the program. For each error:
2. Specify the line number
3. Clearly describe the nature of the error

(b) Classification of Errors (Understanding + Analysis)

1. Classify each identified error into the appropriate compiler phase:
2. Lexical Errors
3. Syntax Errors
4. Semantic Errors
5. Logical Errors
6. Runtime Errors

(c) Compilation Behaviour (Evaluation Level)

1. Analyse which errors will be detected during compilation and which will only appear during execution.
2. Explain how early-stage errors (lexical/syntax) may prevent detection of later-stage errors.

(d) Code Correction (Application Level)

1. Rewrite the corrected version of the program such that:
2. All errors are removed
3. The program executes correctly
4. Proper syntax and semantics are maintained

(e) Impact Analysis (Evaluation Level)

1. Discuss the impact of at least three errors on program execution.
2. Explain how incorrect constructs like improper pointer usage, division by zero, and infinite loops affect system behaviour.

(f) Advanced Compiler Perspective (Create Level)

1. Suggest how a compiler can be improved to:
2. Provide better error messages
3. Perform early detection of runtime errors
4. Propose at least two enhancements in compiler design for efficient debugging.

Q2. The following C program contains **multiple lexical errors** introduced intentionally. These errors occur due to **invalid tokens, incorrect identifiers, malformed constants, and illegal symbols**, which are typically detected during the **lexical analysis phase** of a compiler. Carefully examine the program and answer the questions given below.

```
#include <stdio.h>
```

```
int main() {  
    int lnum = 100;  
    float rate@ = 25.5;  
    char ch = 'abc';  
}
```

```
int total = 50$20;  
int num#1 = 30;  
int %count = 40;
```

```
printf("Welcome to Compiler Design);
```

```
int x = 09;  
int y = 0xZ12;  
int z = 12.3.4;
```

```
char str1[] = "Hello;  
char str2[] = "World";
```

```
int _valid = 10;  
int invalid-id = 20;
```

```
float value = 5..6;  
int data = 10@5;
```

```
char c = 'xy';  
int 3value = 60;
```

```
printf("Value: %d\n", total);
```

```
return 0;  
}
```

Tasks

(a) Error Identification

- i) Identify all lexical errors in the program.
- ii) List the line number and the invalid token.

(b) Error Classification

- i) For each error, explain why it is considered a lexical error.
- ii) Specify which rule of token formation is violated:
- iii) Identifier rules
- iv) Constant formation
- v) String/character literal rules
- vi) Operator/symbol rules

(c) Token Correction

- i) Rewrite each invalid token into a valid token.
- ii) Example: 2value → value2

(d) Compiler Behaviour Analysis

- i) Explain how a lexical analyzer (scanner) processes such errors.
- ii) Discuss whether the compiler:
- iii) Stops immediately
- iv) Continues scanning using error recovery

(e) Advanced Thinking

- i) Design a simple rule (or DFA idea) to detect invalid identifiers starting with digits.
- ii) Suggest how error messages can be improved for beginners.

Q3. The following C program is designed to perform arithmetic operations, function calls, conditional checks, pointer usage, and looping. However, the program contains **multiple errors distributed across different phases of a compiler**, including:

- A. Lexical Errors
- B. Syntax Errors
- C. Semantic Errors
- D. Intermediate Code Generation Issues (logic/precedence)
- E. Code Optimization Issues (redundancy/inefficiency)
- F. Runtime / Target Code Errors

Carefully analyse the program and answer the questions below.

```
#include <stdio.h>
```

```
int divide(int a, int b) {  
    return a / b  
}
```

```
int main() {  
    int #value = 20;  
    float num = 15.5  
    int result;
```

```
    result = divide(10);
```

```
    if(result = 5) {  
        printf("Result is 5\n")  
    }
```

```
    int x = 10;  
    int y = 0;  
    int z = x / y;
```

```
    int arr[4] = {1,2,3,4};  
    printf("%d\n", arr[4]);
```

```
    int *ptr;  
    *ptr = 25;
```

```
    int total;  
    total = x + y * result + 0;
```

```
    while(x < 20) {
```

```
printf("%d\n", x);  
}
```

```
return 0;  
}
```

Do the following Tasks

(a) Identify Errors (Analysis Level)

- I. Identify **all errors** in the program.
- II. Mention **line number + error description**.

(b) Classify by Compiler Phase

Classify each error into:

- I. Lexical (invalid symbols like @num)
- II. Syntax (missing semicolon, brackets)
- III. Semantic (function argument mismatch)
- IV. Intermediate Code Issues (wrong expression handling/precedence)
- V. Optimization Issues (redundant + 0)
- VI. Runtime Errors (division by zero, invalid pointer access)

(c) Compilation vs Execution (Evaluation)

- I. Which errors are detected at **compile time**?
- II. Which errors appear only during **execution**?
- III. Explain why some errors prevent further compilation.

(d) Correct the Program (Application)

- I. Rewrite the **fully corrected program**.
- II. Ensure:
 - A. Proper syntax
 - B. Correct semantics
 - C. Safe execution

(e) Deep Analysis (Evaluation)

- I. Explain how **incorrect pointer usage** affects runtime.
- II. Analyse how **array bounds violation** impacts memory.
- III. Discuss how **operator precedence errors** affect intermediate code.

(f) Advanced Task (Create Level)

- I. Suggest **2 improvements** in compiler design to:
 - A. Detect runtime errors earlier
 - B. Improve debugging messages

II. Propose how **optimization phase** can remove redundant expressions.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyse and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS

Q1 (a) Error Identification

Code Line	Error	Description
for(i = 0; i <= n; i++)	Incorrect loop condition	The condition $i \leq n$ causes one extra iteration, resulting in access to <code>arr[n]</code> , which is outside the array bounds. It should be $i < n$.
int arr[5] = {1,2,3,4,5}	Missing semicolon	A semicolon (;) is required at the end of the declaration. This is a syntax error.
total = sumArray(arr);	Function argument mismatch	The function <code>sumArray()</code> requires two arguments (<code>arr</code> and <code>n</code>), but only one argument is passed.
printf("Correct Sum\n")	Missing semicolon	The <code>printf()</code> statement should end with a semicolon.
int c = a / b;	Division by zero	Since <code>b</code> is initialized to 0, this statement causes a division-by-zero error during execution.
int *p;	Uninitialized pointer	The pointer <code>p</code> is declared but not initialized to a valid memory address.
*p = 20;	Invalid pointer dereference	Dereferencing an uninitialized pointer results in undefined behaviour and may cause a segmentation fault.

(b) Classification of Errors

Code Line	Compiler Phase	Reason
int arr[5] = {1,2,3,4,5}	Syntax Error	Missing semicolon violates C syntax rules.
total = sumArray(arr);	Semantic Error	Function call does not match the required parameter list.
printf("Correct Sum\n")	Syntax Error	Missing semicolon after the <code>printf()</code> statement.
for(i = 0; i <= n; i++)	Logical Error	The loop executes one extra iteration, causing out-of-bounds array access.
int c = a / b;	Runtime Error	Division by zero occurs during execution.
int *p; and *p = 20;	Runtime Error	Using an uninitialized pointer causes invalid memory access.

Explanation:**1. Syntax Errors**

Syntax errors violate the grammar rules of the C language. Missing semicolons prevent the compiler from correctly parsing the program.

2. Semantic Errors

The function `sumArray()` expects two parameters (`arr` and `n`), but only one is supplied. The syntax is correct, but the meaning is incorrect, so the compiler reports an argument mismatch.

3. Logical Errors

The condition `i <= n` causes the loop to execute one extra iteration. Although the program compiles, it produces incorrect behaviour by accessing memory outside the array.

4. Runtime Errors

Division by zero and dereferencing an uninitialized pointer occur only when the program executes. These errors may cause the program to terminate abnormally.

5. Lexical Errors

There are no lexical errors in this program because all identifiers, keywords, literals, and symbols are valid.

(c) Compilation Behaviour (Evaluation Level)**(i) Errors detected during compilation**

- Missing semicolon after array declaration.
- Missing semicolon after `printf()`.
- Incorrect number of arguments in `sumArray()`.

These errors stop successful compilation because the compiler cannot generate executable code.

(ii) Errors detected during execution

- Division by zero.
- Dereferencing an uninitialized pointer.
- Array out-of-bounds due to incorrect loop condition.

These errors are discovered only when the affected statements execute.

(iii) Why early-stage errors prevent later-stage error detection

Compilation occurs in multiple phases. The compiler first checks lexical and syntax correctness before performing semantic analysis and generating executable code. If syntax errors such as missing semicolons exist, compilation stops immediately. As a result, later errors like division by zero or invalid pointer access cannot be detected because no executable program is produced.

(d) Corrected Program

```
#include <stdio.h>
int sumArray(int arr[], int n)
{
```

```

int i, sum = 0;
for(i = 0; i < n; i++)
{
    sum += arr[i];
}
return sum;
}
int main()
{
    int arr[5] = {1,2,3,4,5};
    int total;
    total = sumArray(arr, 5);
    if(total == 15)
    {
        printf("Correct Sum\n");
    }
    int a = 10;
    int b = 2;
    int c = a / b;
    printf("Division Result = %d\n", c);
    int value = 20;
    int *p = &value;
    *p = 20;
    printf("Pointer Value = %d\n", *p);
    return 0;
}

```

Justification

- Added missing semicolons.
- Passed both required arguments to sumArray().
- Changed $i \leq n$ to $i < n$ to prevent out-of-bounds access.
- Replaced zero divisor with a valid value.
- Properly initialized the pointer before dereferencing.

The corrected program compiles successfully and executes safely.

(e) Impact Analysis

1. Improper Pointer Usage

An uninitialized pointer stores an unpredictable memory address. Writing through it may overwrite important memory or cause a segmentation fault, making the program crash unexpectedly.

2. Division by Zero

Division by zero is undefined in C. It usually results in a runtime exception and immediate program termination. Such errors reduce software reliability.

3. Incorrect Loop Condition

Using $i \leq n$ instead of $i < n$ accesses memory beyond the array boundary. This may produce incorrect output, corrupt nearby memory, or crash the program depending on the system.

(f) Advanced Compiler Perspective (Create Level)

(i) How can a compiler provide better error messages?

- Display the exact line number and the specific reason for the error.
- Suggest possible corrections, such as:
 - *"Did you forget a semicolon?"*
 - *"Function 'sumArray' expects 2 arguments but 1 was provided."*
- Highlight the incorrect statement to help programmers locate errors quickly.
-

(ii) How can runtime errors be detected earlier?

Modern compilers can perform additional static analysis to identify potential runtime problems before execution, such as:

- Possible division by zero.
- Use of uninitialized pointers.
- Array index exceeding declared bounds.
- Access to invalid memory locations.

(iii) Two enhancements for efficient debugging

1. Static Code Analysis

The compiler can analyse variable initialization, pointer safety, and array bounds before generating machine code. This helps detect many runtime issues during compilation.

2. Intelligent Warning System

The compiler can issue detailed warnings for suspicious constructs, such as incorrect loop conditions, unused variables, unreachable code, and unsafe pointer operations. These warnings help developers fix problems before execution.

Conclusion

This program contains syntax, semantic, logical, and runtime errors, while no lexical errors are present. After correcting the syntax, passing proper function arguments, fixing the loop condition, preventing division by zero, and initializing the pointer correctly, the program compiles successfully and executes safely. This demonstrates how different phases of a compiler work together to detect and prevent errors.

Q2 (a) Error Identification

(i & ii) Identify all lexical errors with the invalid token

Code Line	Invalid Token	Lexical Error
int lnum = 100;	lnum	Identifier starts with a digit.
	Invalid	

Code Line	Token	Lexical Error
float rate@ = 25.5;	rate@	@ is not allowed in identifiers.
char ch = 'abc';	'abc'	Character literal contains more than one character.
int total = 50\$20;	\$	\$ is an invalid symbol in C.
int num#1 = 30;	num#1	# is not permitted in identifiers.
int %count = 40;	%count	Identifier cannot begin with %.
printf("Welcome to Compiler Design);	Missing closing "	Unterminated string literal.
int x = 09;	09	Invalid octal constant (9 is not allowed in octal).
int y = 0xZ12;	0xZ12	Invalid hexadecimal constant (Z is not a hexadecimal digit).
int z = 12.3.4;	12.3.4	Malformed numeric constant with multiple decimal points.
char str1[] = "Hello;	Missing closing "	Unterminated string literal.
int invalid-id = 20;	invalid-id	- is interpreted as the subtraction operator, making the identifier invalid.
float value = 5..6;	5..6	Invalid floating-point constant.
int data = 10@5;	@	@ is an illegal symbol in C.
char c = 'xy';	'xy'	Character literal contains multiple characters.
int 3value = 60;	3value	Identifier starts with a digit.

(b) Error Classification

Explain why each is a lexical error and the violated rule

Code Line	Violated Rule	Reason
int 1num = 100;	Identifier Rule	Identifiers cannot begin with digits.
float rate@ = 25.5;	Identifier Rule	Only letters, digits and _ are allowed.
char ch = 'abc';	Character Literal Rule	A character constant should contain only one character.
int total = 50\$20;	Operator/Symbol Rule	\$ is not a valid operator or symbol.
int num#1 = 30;	Identifier Rule	# is not permitted inside identifiers.
int %count = 40;	Identifier Rule	Identifiers cannot begin with %.
printf("Welcome to Compiler Design);	String Literal Rule	String literals must end with a closing quotation mark.
Code Line	Violated Rule	Reason

int x = 09;	Constant Formation Rule	Octal numbers use digits 0–7 only.
int y = 0xZ12;	Constant Formation Rule	Hexadecimal numbers allow only 0–9 and A–F.
int z = 12.3.4;	Constant Formation Rule	Numeric constants can contain only one decimal point.
char str1[] = "Hello;	String Literal Rule	Missing closing quotation mark.
int invalid-id = 20;	Identifier Rule	Hyphen is treated as the subtraction operator.
float value = 5..6;	Constant Formation Rule	Floating-point constants cannot contain two decimal points.
int data = 10@5;	Operator/Symbol Rule	@ is not a valid C operator.
char c = 'xy';	Character Literal Rule	Character literals must contain exactly one character.
int 3value = 60;	Identifier Rule	Identifiers cannot start with numbers.

(c) Token Correction

Invalid Token	Correct Token
lnum	num1
rate@	rate
'abc'	'a'
50\$20	50 + 20
num#1	num1
%count	count
"Welcome to Compiler Design	"Welcome to Compiler Design"
09	9 (or 011 if octal is intended)
0xZ12	0xA12
12.3.4	12.34
"Hello	"Hello"
invalid-id	invalid_id
5..6	5.6
10@5	10 + 5
'xy'	'x'
3value	value3

(d) Compiler Behaviour Analysis

(i) How does the lexical analyser process these errors?

The lexical analyser (scanner) reads the source program character by character and groups them into valid tokens such as identifiers, keywords, constants, operators, and literals. Whenever it encounters an invalid token, it reports a lexical error because the token does not follow the lexical rules of the C language.

For example:

- 1num violates the identifier formation rule.
- 0xZ12 is recognized as an invalid hexadecimal constant.
- An unterminated string literal is detected when the closing quotation mark is missing.

(ii) Does the compiler stop immediately?

Modern compilers generally do not stop after the first lexical error. Instead, they use error recovery techniques to skip the invalid token, synchronize with the remaining input, and continue scanning. This enables the compiler to report multiple lexical errors in a single compilation, helping programmers correct them more efficiently.

(e) Advanced Thinking

(i) DFA idea to detect identifiers starting with digits

A simple DFA can be designed as follows:

- **Start State (S0):**
 - If the first character is a letter (A–Z, a–z) or underscore (_) → move to Valid Identifier State (S1).
 - If the first character is a digit (0–9) → move to Error State (SE).
- **Valid Identifier State (S1):**
 - Accept letters, digits, and underscores repeatedly.
 - If any illegal symbol (@, #, %, etc.) is found → move to Error State (SE).
- **Error State (SE):**
 - Reject the token and display an appropriate lexical error message.

This DFA ensures that identifiers always begin with a valid character and contain only permitted characters.

(ii) Suggestions to improve compiler error messages

1. Display the exact invalid token along with its position in the source code.
2. Clearly explain why the token is invalid instead of showing a generic error.
3. Suggest a possible correction, for example:
 - 1num → Did you mean num1?
 - rate@ → Remove '@' from the identifier.
4. Highlight the erroneous token in the editor to make debugging easier.
5. Group similar lexical errors together to reduce confusion for beginners.

Conclusion

The given program contains 16 lexical errors, including invalid identifiers, malformed numeric constants, illegal symbols, and incorrect string and character literals. These errors are detected

during the lexical analysis phase, before syntax analysis begins. Correcting the invalid tokens enables the compiler to proceed with parsing and semantic analysis, resulting in successful compilation.

Q3 (a) Identify Errors

Code Line	Error	Description
return a / b	Missing semicolon	The return statement must end with a semicolon.
int #value = 20;	Invalid identifier	# is not allowed in C identifiers.
float num = 15.5	Missing semicolon	Declaration is missing a semicolon.
result = divide(10);	Function argument mismatch	divide() requires two arguments, but only one is passed.
if(result = 5)	Assignment instead of comparison	= assigns a value instead of comparing. It should be ==.
printf("Result is 5\n")	Missing semicolon	printf() statement should end with ;.
int z = x / y;	Division by zero	Since y = 0, the program attempts to divide by zero during execution.
printf("%d\n", arr[4]);	Array index out of bounds	The array has valid indices 0–3. Accessing arr[4] is invalid.
int *ptr;	Uninitialized pointer	The pointer is declared but not initialized.
*ptr = 25;	Invalid pointer dereference	Dereferencing an uninitialized pointer causes undefined behaviour.
total = x + y * result + 0;	Redundant expression	Adding +0 has no effect and is an optimization issue.
while(x < 20)	Infinite loop	x is never incremented inside the loop, so the condition never becomes false.

(b) Classify by Compiler Phase

Code Line	Compiler Phase	Reason
int #value = 20;	Lexical Error	# is an invalid character in an identifier.
return a / b	Syntax Error	Missing semicolon violates C grammar.
float num = 15.5	Syntax Error	Missing semicolon after declaration.
printf("Result is 5\n")	Syntax Error	Missing semicolon after printf().
result = divide(10);	Semantic Error	Incorrect number of function arguments.
if(result = 5)	Intermediate Code Issue	Assignment changes the program logic; comparison (==) was intended.
Code Line	Compiler Phase	Reason

total = x + y * result + 0;	Optimization Issue	+0 is redundant and can be removed during optimization.
int z = x / y;	Runtime Error	Division by zero occurs during execution.
printf("%d\n", arr[4]);	Runtime Error	Accesses memory outside the array boundary.
*ptr = 25;	Runtime Error	Writes to an invalid memory location.
while(x < 20)	Runtime/Logical Error	The loop never terminates because x is not updated.

(c) Compilation vs Execution (Evaluation)

(i) Errors detected at compile time

- Invalid identifier (#value)
- Missing semicolons
- Incorrect function call (divide(10))
- Assignment inside the if condition (usually reported as a warning)

These errors are identified before executable code is generated.

(ii) Errors detected during execution

- Division by zero
- Array index out of bounds
- Uninitialized pointer dereference
- Infinite loop

These errors occur only when the corresponding statements are executed.

(iii) Why do some errors prevent further compilation?

Compilation follows several phases such as lexical analysis, syntax analysis, and semantic analysis. If lexical or syntax errors are found, the compiler cannot correctly understand the structure of the program. As a result, code generation is stopped, preventing later phases from analysing the remaining program.

(d) Corrected Program (Application Level)

```
#include <stdio.h>
int divide(int a, int b)
{
    if (b == 0)
    {
        printf("Division by zero is not allowed.\n");
        return 0;
    }
    return a / b;
}
```

```

int main()
{
    int value = 20;
    float num = 15.5;
    int result;
    result = divide(10, 2);
    if(result == 5)
    {
        printf("Result is 5\n");
    }
    int x = 10;
    int y = 2;
    int z = x / y;
    printf("Division Result = %d\n", z);
    int arr[4] = {1,2,3,4};
    printf("%d\n", arr[3]);
    int data = 25;
    int *ptr = &data;
    *ptr = 30;
    int total;
    total = x + y * result;
    while(x < 20)
    {
        printf("%d\n", x);
        x++;
    }
    printf("Total = %d\n", total);
    return 0;
}

```

Justification

- Removed the invalid identifier.
- Added all missing semicolons.
- Passed the correct number of function arguments.
- Replaced = with == in the if condition.
- Prevented division by zero.
- Corrected the array index.
- Initialized the pointer before dereferencing.
- Removed the redundant +0.
- Incremented x inside the loop to avoid an infinite loop.

(e) Deep Analysis (Evaluation)

(i) Effect of incorrect pointer usage

An uninitialized pointer does not point to a valid memory location. Writing through such a pointer can overwrite important memory, corrupt data, or cause a segmentation fault. Pointer-related bugs are difficult to trace and often lead to unpredictable program behaviour.

(ii) Impact of array bounds violation

Arrays in C are not automatically bounds-checked. Accessing `arr[4]` when the array size is only four elements results in reading memory outside the allocated array. This may produce incorrect output, corrupt adjacent memory, or cause the program to crash.

(iii) Effect of operator precedence on intermediate code

The statement:

```
if(result = 5)
```

assigns the value 5 to result instead of comparing it. During intermediate code generation, the compiler generates instructions for assignment rather than comparison, causing the condition to always evaluate as true. Consequently, the program follows incorrect control flow.

(f) Advanced Task

(i) Two improvements in compiler design

1. Early Runtime Error Detection

The compiler can use static analysis techniques to detect possible runtime problems before execution, such as:

- Division by zero
- Array index violations
- Use of uninitialized pointers
- Potential infinite loops

This helps programmers fix issues before running the program.

2. Improved Debugging Messages

Compilers should provide descriptive error messages with:

- Exact location of the error
- Reason for the error
- Suggested correction
- Highlighted source code

For example:

Error: Function `divide()` expects 2 arguments but only 1 was provided.

Suggestion: Call `divide(10, 2)` instead.

Such messages make debugging easier, especially for beginners.

(ii) How can the optimization phase remove redundant expressions?

The optimization phase improves efficiency by removing unnecessary operations. For example:

```
total = x + y * result + 0;
```

Since adding zero does not change the value, the compiler simplifies it to:

```
total = x + y * result;
```

Similarly, the optimizer can eliminate dead code, remove unused variables, simplify constant expressions, and reduce unnecessary calculations, resulting in faster and more efficient programs.

Conclusion

The given program contains lexical, syntax, semantic, intermediate code, optimization, and runtime errors. Some errors, such as invalid identifiers and missing semicolons, are detected during compilation, while others, including division by zero, array bounds violations, invalid pointer access, and infinite loops, occur only during execution. After correcting these issues, the program compiles successfully, executes safely, and follows proper C programming practices.